Intro and Rust

CS 1622

Jarrett Billingsley

Class announcements

- **be sure to check the notes on each of my slides**
  - for extra explanations, examples, snarky comments etc.
  - as well as answers to the questions on the slides so you can study.
- also, **be sure to use Powerpoint to view my slides**
  - *not* Keynote, *not* Google Drive; they absolutely butcher alignment and diagrams and arrows and I use a lot of those. it's bad enough that diagrams can become incorrect/misleading.
  - if you use a note-taking app, **open the pptx in Powerpoint and export it as PDF, then use that.** it will look *much* better than opening the pptx directly in your note-taking app.

Administrivia

hi

- you can just call me Jarrett
- **jarrett@cs.pitt.edu**
- **sites.pitt.edu/~jfb42**
- **Office hours:** Mon/Wed 1:15-3:30 in 6509 SENSQ
- **Religious absences:** contact me ASAP
- **Students with disabilities:** contact the DRS ASAP
- **Everything in this section is also on my site in "course info"!**
  - *I use red on the slides for really important things*

Communication

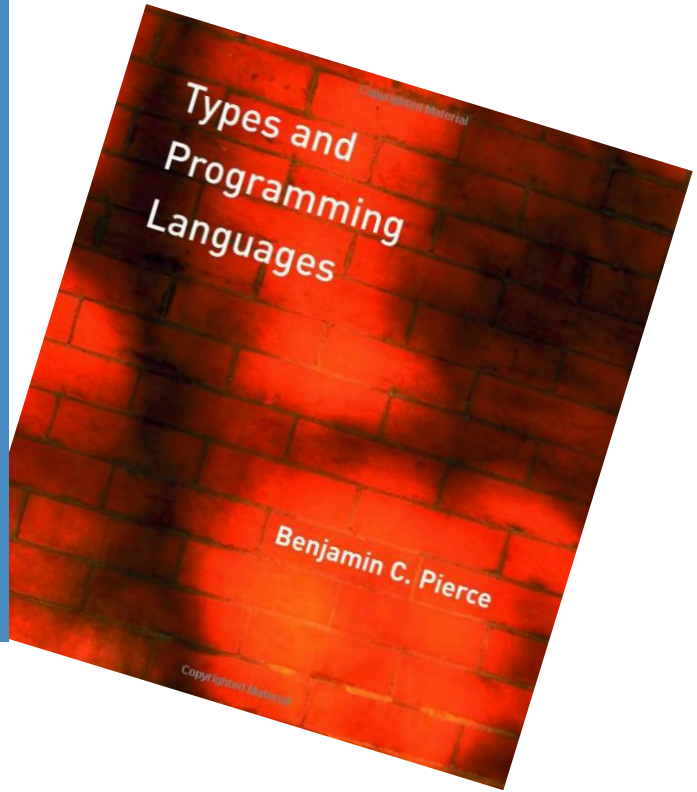
- I send announcements through Canvas which come thru email
  - it is not my responsibility to make sure you get them, it's *yours*
- announcements/grades are the **only** things I use Canvas for
  - (well recording class too)
  - everything else is on my site.
  - including the course schedule/syllabus.
- **Discord hours:** most days after 12PM and before 9PM EDT
  - but I don't really have a healthy work/life separation so
- **For more "official" communications, or if you need my attention *immediately,* use email.**
  - otherwise, Discord is preferred – much better for sharing code, screenshots, etc. in a much quicker way.

Textbooks???

- I'm not really a book person.
- none of these are required and the first two are free (see course info)







Grading

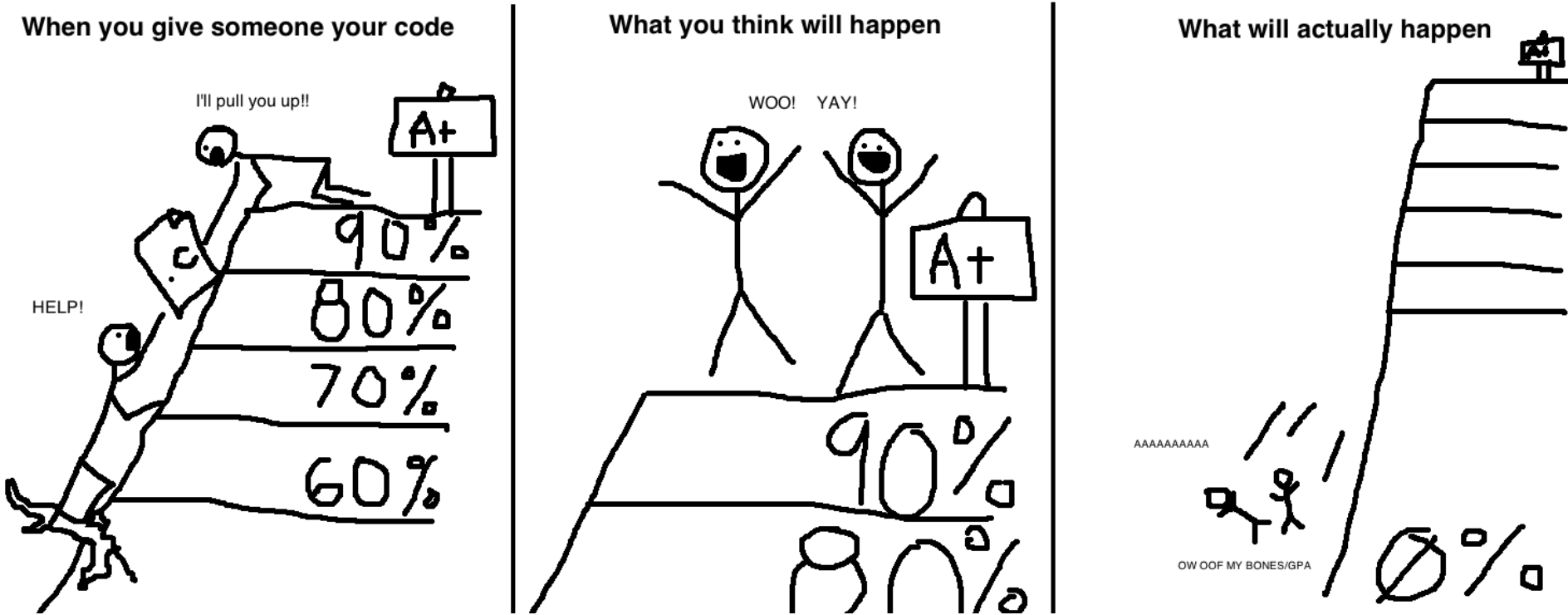
- **Projects (x4): 60%**
  - each is worth 15%
  - **there is a late submission policy,** see the course info page
- **Exams (x2): 40%**
  - each worth 20%
  - **exam 2 is *not* cumulative!** it's just on the stuff after exam 1.
  - on both exam days, we will do a short review session beforehand.
- There are no labs or recitations or anything.
- Attendance is **not graded** for lectures…
  - but **people who don't come and interact don't do well.**

Teaching philosophy

- I'm more of a "big picture" teacher
  - I care about high-level concepts and problem-solving
  - you won't get exam questions about function parameters
- You are a student. ***you are supposed to be confused.***
  - **I wanna help you understand!**
  - never say "this is a dumb question" cause there aren't any!
  - **DON'T STRUGGLE IN SILENCE!!!! EVER!!**
- What is a university setting good for today?
  - focus, practice, and *access to people who know stuff*
  - **your friends do not know as much about this stuff as I do**
  - but also... your friends **can** be very helpful when studying or in practical matters (installing stuff etc)!
  - **studying together is not cheating! do it!**

I can tell when you cheat :^)

- **If you cheat, you fail the course. Period.**
- **Don't post your code on github, baidu etc.**
- **Don't "help" your friends by giving them code.**



Academic Integrity

- If you "help" a friend, **you are only delaying their failure until a later point** when it will be too late to fix the problems.
  - a 0 will hurt way, *way* more than a 60, and it'll hurt you **both.**
- **Don't cheat if your enrollment is contingent upon your GPA.**
  - or your ROTC or your student visa or your graduation or whatever.
- Generally speaking, I trust you!
  - I **try** to be accommodating about extensions and such
  - I don't think most cheaters are being lazy
- But you're an adult, and are **responsible for your actions.**
  - I *hate* being in a position where I have your future in my hands…
  - **especially when you're the one who put it there.**
  - ***Do not take advantage of my trust.***
- If you're confused, **don't cheat, ask for help. yes, even on the exams. yes, even an hour before the project is due.**

Intro

What, Why, and How

- for every topic, ask yourself (and me!) these questions:

Why do we use it?

"splitting code into functions makes it easier to read, understand, and reuse."

What is it?

"a function is a named piece of code."

How do we do it?

"in Java, you write the return type, then the name, then a left parenthesis, then any arguments, with the type and name, separated by commas, then a right parenthesis, then…

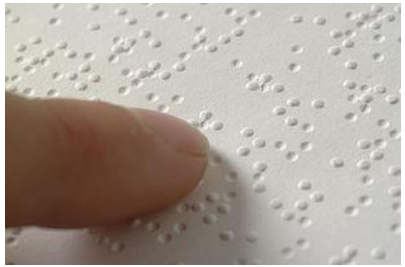
what and **why** are *almost always* more important than **how.**

on the exams, **be sure you are answering the right question.**
if I ask "**what** is X," *don't* explain **how** to X.

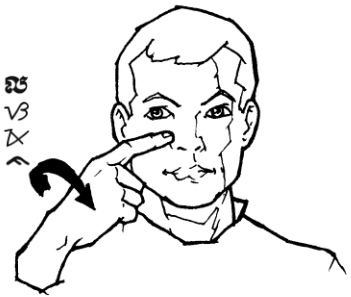
What is language?

- a system of encoding **information** by using **arbitrary symbols**
  - "symbols" can also include sounds, objects, actions, etc.











human languages are **massively** complex, like anything involving brains.

within the mess, there are kernels of mathematical and logical truth…

$$\exists_{x \in S} \forall_{y \neq x} \; P(y) \rightarrow Q(x)$$

$$\int_0^n \frac{x^2}{9}$$

…upon which **computer languages** were designed.

Computer languages

- modeled after human languages, these are much simpler languages used to encode information in a way that is useful for computing.



programming languages encode instructions for the computer to execute.



declarative languages encode data and rules.



interchange and binary file formats encode domain- or application-specific data.



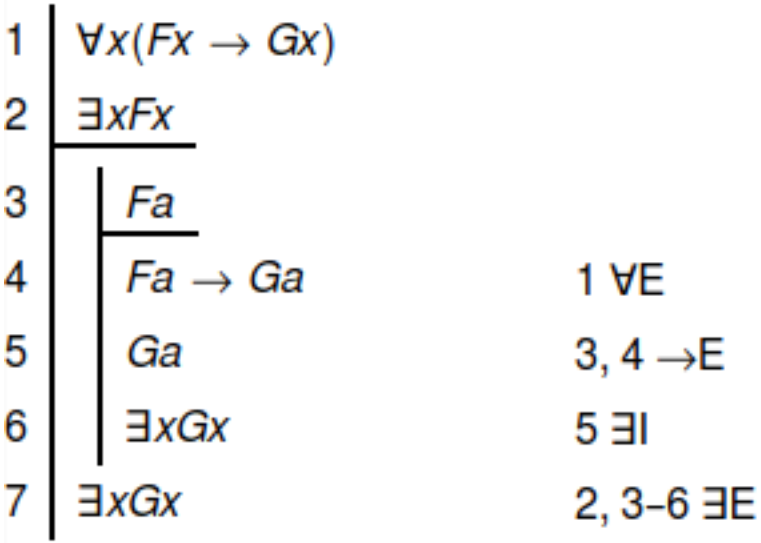
even things like **network protocols** are a kind of language!

Programming languages

- *one* definition is that programs are **sequences of instructions…**
- but **programs are also proofs.**

```
fn main() {
    for i in 0 .. 10 {
        println!("i = {}", i);
    }
}
```

the Curry-Howard correspondence shows that programs and proofs are **two ways of expressing the same mathematical objects.**



there are **proof languages** and **proof assistants** which can function as both programming languages *and* proof checkers.

The Origin of (high-level programming) Languages

- as you learned in 447, **CPUs execute machine code…**
  - and **assembly language** is a textual representation of that.

```
main:
  li   s0, 0
_loop:
  la   a0, msg
  li   v0, 4
  syscall
  move a0, s0
  li   v0, 1
  syscall
  li   a0, '\n'
  li   v0, 11
  syscall
  add  s0, s0, 1
  blt  s0, 10, _loop
```

but even the simplest tasks take a *lot* of code in assembly, and **more code = more mistakes.**

we invented **high-level languages (HLLs)** to let us write programs in shorter, human-friendly ways.

```
fn main() {
  for i in 0 .. 10 {
    println!("i = {}", i);
  }
}
```

Performance ∝ 1 / Abstraction

- HLLs allow us to focus **more on solving problems** than on holding the CPU's hand through every step of computation
- but *most* abstractions come at a **performance cost**

```
for(i = 0; i < n; i++) {
  A[i] = B[i] + C[i];
}
```

```
A = B + C
```

lower-level languages like C provide abstractions that better match* what the underlying hardware provides.

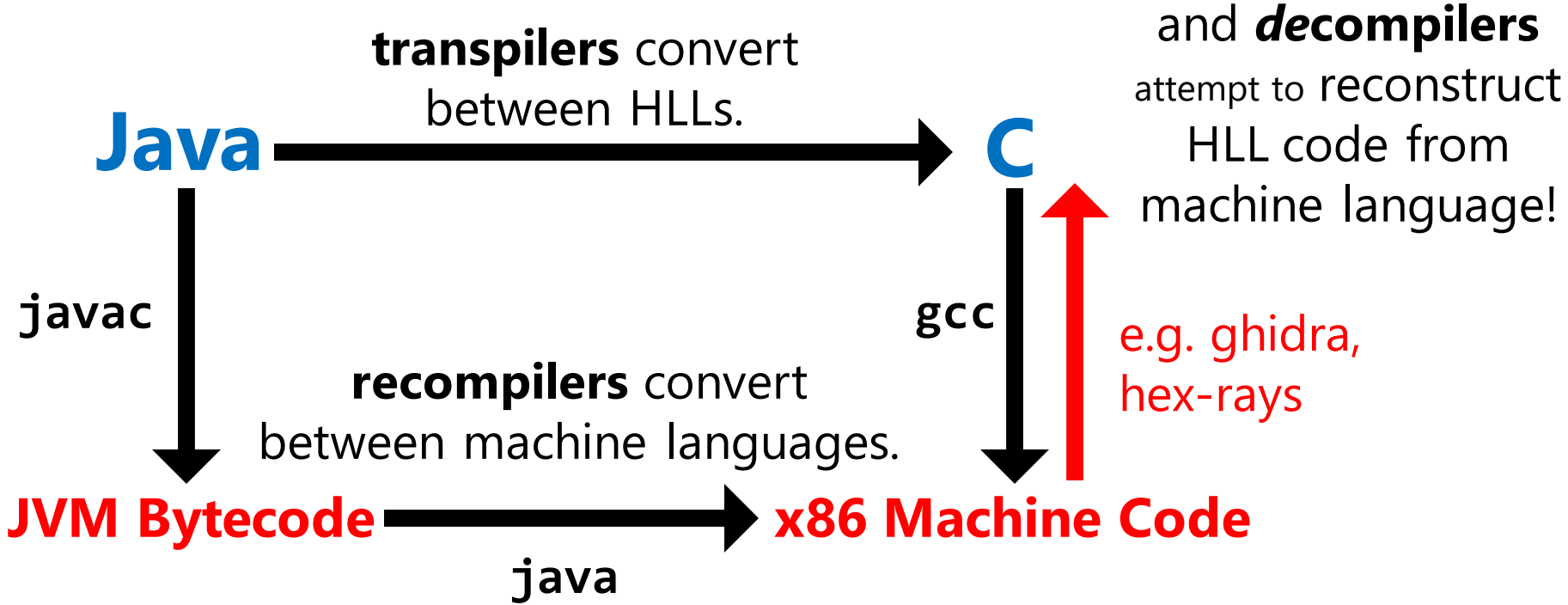
higher-level languages like Python give you more powerful abstractions, at the cost of lower performance (usually).

but it all depends on the CPU design and the *quality of the language implementation*.

Compilers

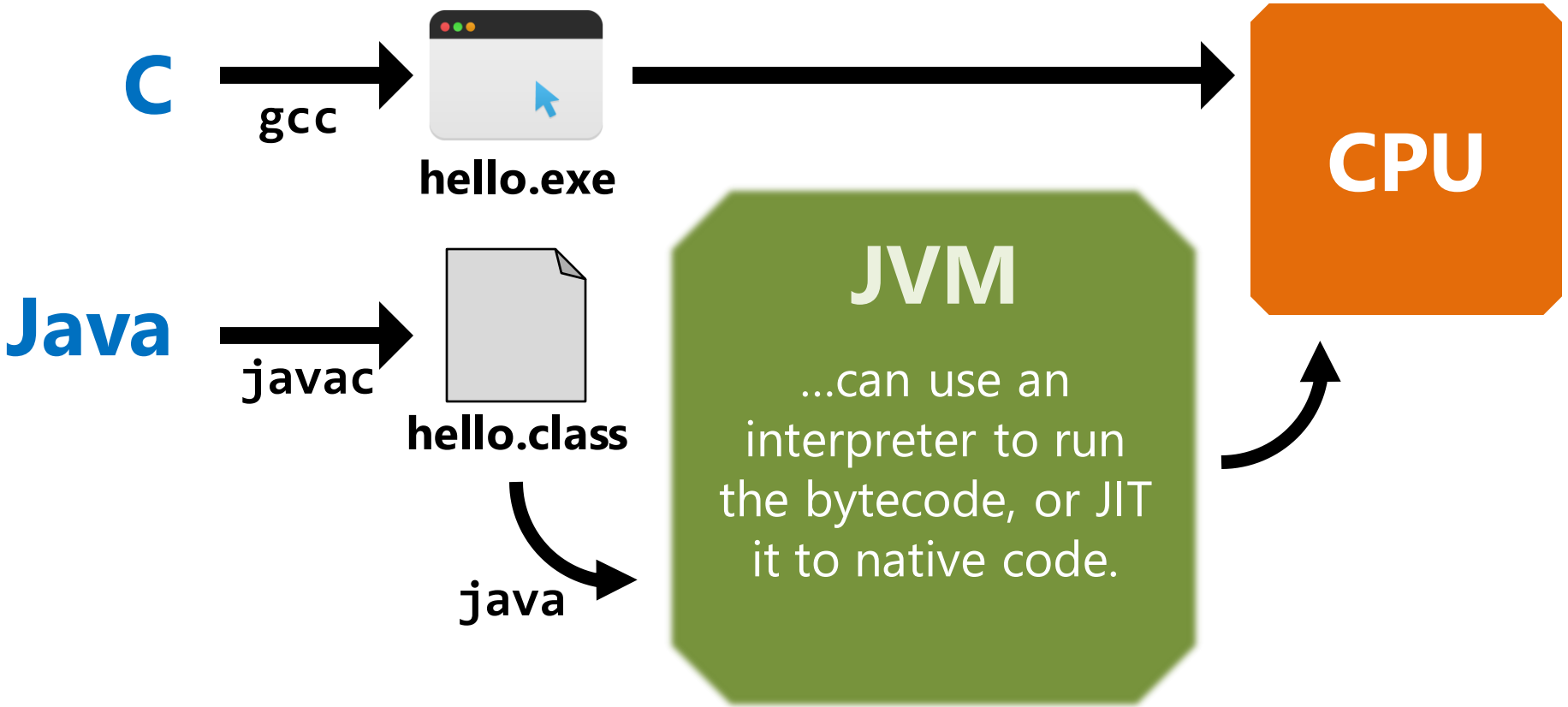
What's a compiler?

- it's a program that **translates** one programming language (the **source** language) to another (the **target** language).
- *typically* this is from an **HLL** to a **machine language.**



Ahead-of-time vs. Just-in-time (AOT vs JIT)

- **AOT** compilers produce a file that contains the native machine code.
- **JIT** compilers produce native machine code *right before it's needed*.



Okay, okay: HOW does a compiler WORK

- there are kind of two broad stages:

the **frontend** reads the source code and checks it for "correctness."

the **backend** produces target code from the compiler's "idea" of your program.



this "idea" is the **intermediate representation (IR):** a sort of *third* language which "bridges the gap."

Frontend step 1: Lexical Analysis (Lexing (or Scanning))

- **lexing** splits the source text into **tokens:** words, symbols, etc.
- it's a straightforward string processing algorithm.

This is English. → `This, is, English, .`

`void main(){}` → `void, main, (, ), {, }`

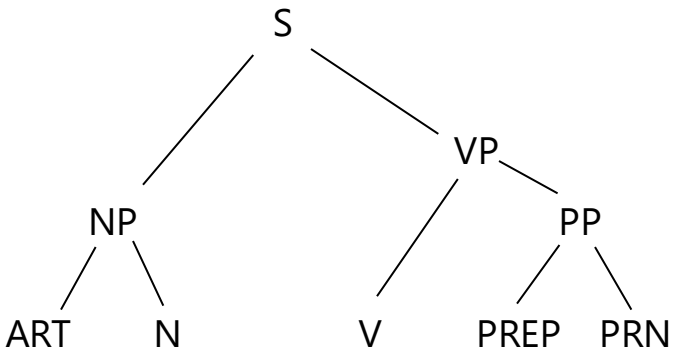
`"hello!` → **error:** unclosed string literal

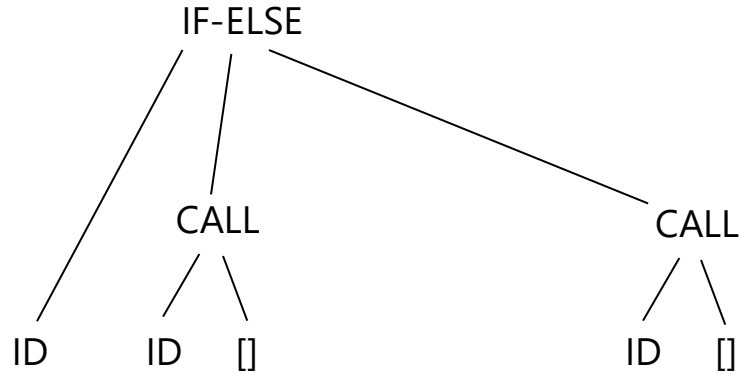
`if<class)++` → `if, <, class, ), ++`

this last one is nonsense, but the lexer doesn't know that.
that's the responsibility of...

Frontend step 2: Syntactic Analysis (Parsing)

- **parsing** takes the lexed tokens and extracts **structure** from them.
- a language's **grammar** defines the rules of these structures.





The cat stares at me.

```
if(x) y(); else z();
```

this is where you get syntactic errors, like "missing semicolon" or "unexpected closing paren" or whatever.

the result is an **AST: Abstract Syntax Tree.**
but this just *looks* like a program. is it one?

Frontend step 3: Semantic Analysis

- this checks for "correctness" according to the rules of the language.
  - things like type checking, name usage, privacy…

Colorless green dreams sleep furiously.

(syntactically correct, but nonsense.)

```
x = 10;
```
 **error:** undefined reference to 'x'

```
int x = 1.0;
```
 **error:** possible loss of precision

```
int x = 1;
```
 

I keep quoting "correctness" because it's *absolutely* possible to write **incorrect programs** that pass these checks.

much PL research is about making these checks better!

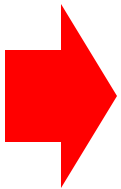
Backend: optimization and code generation

- **optimization** rewrites the program to do the same things but faster.
- **codegen** produces the target language code from the IR.

```
for(i = 0; i < 4; i++)
  A[i] = B[i] + C[i];
```



```
A[0] = B[0] + C[0];
A[1] = B[1] + C[1];
A[2] = B[2] + C[2];
A[3] = B[3] + C[3];
```



```
la   t0, A
la   t1, B
la   t2, C
lw   t3, 0(t1)
lw   t4, 0(t2)
add  t3, t3, t4
sw   t3, 0(t0)
lw   t3, 4(t1)
lw   t4, 4(t2)
add  t3, t3, t4
sw   t3, 4(t0)
lw   t3, 8(t1)
lw   t4, 8(t2)
add  t3, t3, t4
sw   t3, 12(t0)
lw   t3, 12(t1)
lw   t4, 12(t2)
add  t3, t3, t4
sw   t3, 12(t0)
```

and boom… we have a program!

What we'll talk about along the way

- **grammars** and **abstract syntax trees!**
- **type theory** and **type systems!**
  - **static** and **dynamic** typing!
  - **strong** and **weak** typing!
  - **parametric types** and **generics!**
- **runtime** representation!
- **memory management!**
- **ABIs** and **linking!**
- ***and more!***

Rust

If you want to write a compiler...

- different problems require **different tools.**
- when writing compilers, some language features will be very useful:

	Java	C	Rust
Good string manipulation	4/5	0/5	5/5
Automatic memory management	5/5	0/5	5/5
Generics (types and code)	4/5	1/5	4/5
Functional programming style	3/5	0/5	4/5
Algebraic data types	1/5	1/5	5/5
Less repetitive boilerplate	0/5	0/5	4/5
Strong, expressive type system	2/5	0/5	5/5

ratings are my opinion based on experience. this is a limited view of their strengths/weaknesses.

What is Rust?

- it's a **systems language** like C, but designed to not be terrible.
- it's rigorously defined and borrows many features from PL theory…
- …but also tries to be a practical language for writing real programs.

you might have heard that it's complex and confusing. and it sure can be!

```
fn err_context<'a, O, F>(kind: LexErrorKind, parser: F) ->
  impl Fn(Span<'a>) -> LResult<'a, O>
where
  F: Fn(Span<'a>) -> IResult<Span<'a>, O, LexError>,
```

but it can also be simple, elegant, and readable.

```
fn main() {
  let nums = vec![1, 3, 5, 7, 10];

  for (i, n) in nums.iter().enumerate() {
    println!("nums[{}] = {}", i, n);
  }

  let sum: i32 = nums.iter().sum();
  println!("sum of nums = {}", sum);
}
```

Basic tools and vocabulary

- **rustup** is the tool to install the Rust toolchain.
- **cargo** is the "swiss army knife" for creating and building programs.
  - it lets you create new Rust projects with some basic features.
  - it acts as a build tool like **make.**
  - it lets you run tests, build documentation, and so on.
  - it lets you specify dependencies on external libraries (crates).
  - it automatically downloads, builds, and installs those libraries.
- a **crate** is like a Java **package:** a self-contained program or library.
  - **crates.io** is the website where you can search for crates.
  - each Rust program you make is its own crate.

Rust by Comparison

- here is a simple program written in Java and Rust.

```java
public class Code {
  public static void main(String[] a) {
    for(int i = 1; i <= 100; i++) {
      if((i % 3) == 0)
        System.out.print("Fizz");

      if((i % 5) == 0)
        System.out.print("Buzz");

      if((i % 3) != 0 && (i % 5) != 0)
        System.out.print(i);

      System.out.println();
    }
  }
}
```

```rust
fn main() {
  for i in 1 ..= 100 {
    if (i % 3) == 0 {
      print!("Fizz");
    }
    if (i % 5) == 0 {
      print!("Buzz");
    }
    if (i % 3) != 0 && (i % 5) != 0 {
      print!("{}", i);
    }
    println!();
  }
}
```

what are some differences?

(see slide notes for some answers)

More Comparison

- it really isn't THAT scary. (class/main/System.out omitted for space)

```java
int[] nums = new int[]{1, 3, 5, 7, 10};

for(int i = 0; i < nums.length; i++) {
  printf("nums[%d] = %d\n", i, nums[i]);
}

int sum = Arrays.stream(nums).sum();
printf("sum of nums = %d", sum);
```

```rust
let nums = vec![1, 3, 5, 7, 10];

for (i, n) in nums.iter().enumerate() {
  println!("nums[{}] = {}", i, n);
}

let sum: i32 = nums.iter().sum();
println!("sum of nums = {}", sum);
```

takeaways:
- variables declared with **`let`**
- arrays called "vectors"*
- **`.iter()`** seems to do Fun Iteration Things
- **`i32`** instead of **`int`**
- print formatting done with **`{}`**

oh no I'm not good with computer what does this me

- let's try something.

```
let nums = vec![1, 3, 5, 7, 10];
for n in nums { println!("{}", n); }
for (i, n) in nums.iter().enumerate() {
  println!("nums[{}] = {}", i, n);
}
```

Rust's compiler errors tend to be *very* verbose, but it's because **it actually tries to help you fix the error!**

```
error[E0382]: borrow of moved value: `nums`
  --> src/main.rs:42:16
   |
36 |     let nums = vec![1, 3, 5, 7, 10];
   |         ---- move occurs because `nums` has type `std::vec::Vec<i32>`, which does not implement the `Copy` trait
37 |
38 |     for n in nums {
   |              ----
   |              |
   |              value moved here
   |              help: consider borrowing to avoid moving into the for loop: `&nums`
...
42 |     for (i, n) in nums.iter().enumerate() {
   |                   ^^^^ value borrowed here after move
```

following its help works here, and often does!

What I want you to do!

For next class...

- On the course info page, find the Rust Programming Language book.
- follow along with chapters 1-3 to install and play around with Rust.
  - **Windows users,** use PowerShell or WSL. cmd.exe is dead.
- you're already programmers so a lot of things will be obvious to you!
- try this: *just* read **the code examples** and if you're confused, **read the text before it.** that way you can skip the easy stuff.